

Recursion and the Call Stack

Matteo Fasiolo

Previously

- A function can take input parameters and return a value.
- When a function is called, each parameter is a new **local variable** initialized with a copy of the corresponding argument's value.
- A function call changes the control flow:
 1. execution enters the called function;
 2. the function executes;
 3. execution returns to the caller.
- Loops repeat instructions while reusing the same block of code.

A familiar task: countdown

Today we will introduce **recursion** and the **call stack**.

How can we print 3 2 1?

We already know how to use a loop:

```
for (int n = 3; n >= 1; n = n - 1) {  
    printf("%d\n", n);  
}
```

The loop describes every repetition using a counter.

Solve a smaller version of the same problem

Pseudo-code to count down from n :

1. If $n \leq 0$, stop.
2. Otherwise:
 - print n ;
 - count down from $n - 1$.

Last step solves **the same problem with a smaller input**.

Recursion

A function is **recursive** when it calls itself:

```
void countdown(int n) {  
    if (n <= 0) {  
  
    } else {  
        printf("%d\n", n);  
        countdown(n - 1);  
    }  
    return;  
}
```

```
countdown(3);
```

Note return can be omitted for void functions.

The three parts of simple recursion

```
void countdown(int n) {  
    if (n <= 0) {          // 1. base case  
  
    } else {  
        printf("%d\n", n);  
        countdown(n - 1); // 2. recursive call  
                           // 3. input gets closer to  
                           //      base case  
    }  
}
```

The base case is a version of the problem that we can solve **without another recursive call**.

“Instrumenting” the program

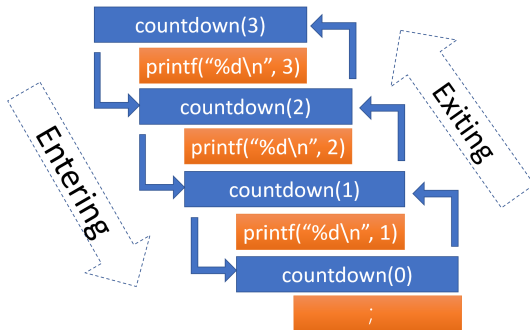
Adding prints to get information on control flow:

```
void countdown(int n) {  
    printf("enter countdown(%d)\n", n);  
  
    if (n <= 0) {  
        printf("base case\n");  
    } else {  
        printf("%d\n", n);  
        countdown(n - 1);  
    }  
  
    printf("leave countdown(%d)\n", n);  
}
```

Calls go down; returns come back up

```
enter countdown(3)
3
enter countdown(2)
2
enter countdown(1)
1
enter countdown(0)
base case
leave countdown(0)
leave countdown(1)
leave countdown(2)
leave countdown(3)
```

Let's look at `countdown_stack.c`.



A question the code does not answer

When `countdown(0)` finishes, the program must remember that it should return to:

```
countdown(1)
```

and later to `countdown(2)`, `countdown(3)` and `main`.

It must also remember the values of `n` belonging to each call.

Where is are these values recorded?

In the **call stack**.

What is a stack?

A **stack** is a last-in, first-out data structure.

push A	push B	push C	pop	pop
A	B	C	B	A
	A	B	A	
		A		

- push: add a new item to the top;
- pop: remove the item at the top.

The call stack uses this to record active function calls.

That is, the set of functions that are still running.

Like an ice cream cone: the last scoop added is the first one eaten.



When a function is called, the program creates a **stack frame** for that call.

A frame records:

- parameter values;
- local-variable values;
- information needed to return to the caller.

When the function returns, its frame is removed.

Example: `countdown(3)`

Immediately after entering `countdown(0)`:

Position	Active call	Its value of <code>n</code>	What it is waiting for
top	<code>countdown(0)</code>	0	Nothing: base case
	<code>countdown(1)</code>	1	<code>countdown(0)</code> to return
	<code>countdown(2)</code>	2	<code>countdown(1)</code> to return
	<code>countdown(3)</code>	3	<code>countdown(2)</code> to return
bottom	<code>main</code>	–	<code>countdown(3)</code> to return

These are four calls of the same function, but **four different frames**.

countdown(0) makes no further recursive call.

What happens to the stack when we exit countdown(0)?

Its frame is removed, so countdown(1) resumes immediately after its recursive call.

```
countdown(0) returns  
countdown(1) returns  
countdown(2) returns  
countdown(3) returns  
main resumes
```

Removing frames as calls return is called **unwinding the stack**.

Local variables and function calls

Important details:

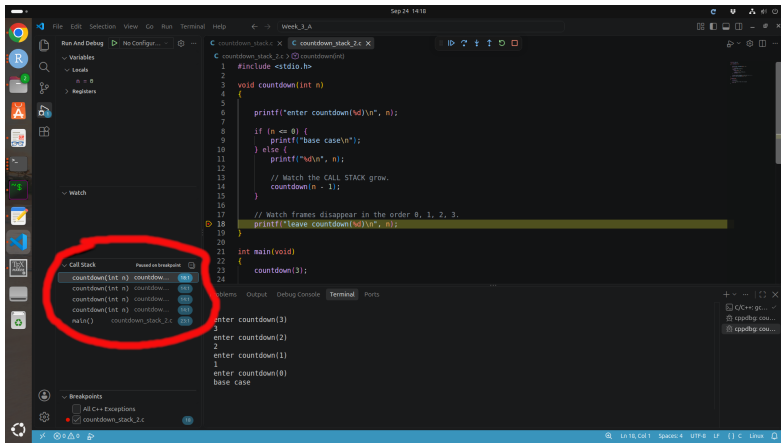
- A function's parameters and the variables declared inside it are **local variables**.
- Their names cannot be used from another function. This is called **local scope**.
- Each active function call has its own parameter and local-variable values.

The same function instructions are reused by every call.

Back to VS Code.

Let's look at `countdown_stack_2.c`

Let's focus on the call stack:



What the debugger confirmed:

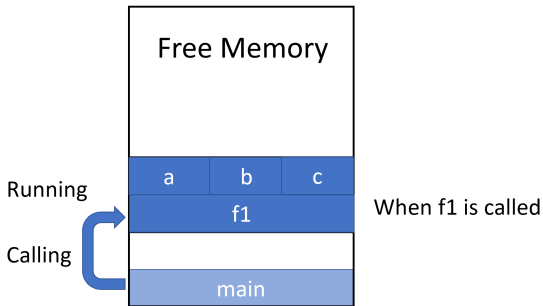
- recursive calls added stack frames;
- every call had its own value of n ;
- the different values of n were stored in different frames;
- returns removed frames in reverse order;

Stack diagrams help visualising the state of the call stack.

Consider a function f1 calling f2, with f2 calling f3.

```
#include <stdio.h>
void f3(){
    int d = 6, e = 7;
}
void f2(){
    int a = 4, b = 5;
    f3();
}
void f1(){
    int a = 1, b = 2, c = 3;
    f2();
}
int main(void){
    f1();
    return 0;
}
```

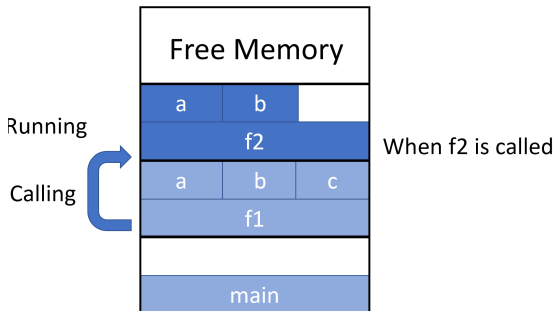
Below is the stack while f1 is called.



where

```
void f1(){  
    int a = 1, b = 2, c = 3;  
    f2();  
}
```

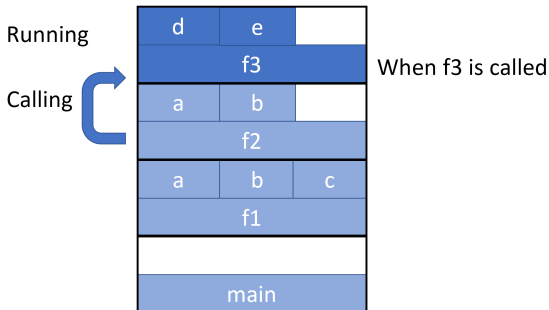
Below is the stack while f2 is called.



where

```
void f2(){  
    int a = 4, b = 5;  
    f3();  
}
```

Below is the stack while f3 is called.



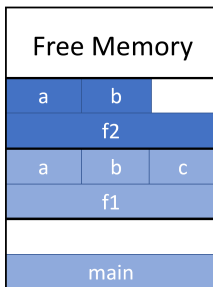
where

```
void f3(){  
    int d = 6, e = 7;  
}
```

Stack Memory Release

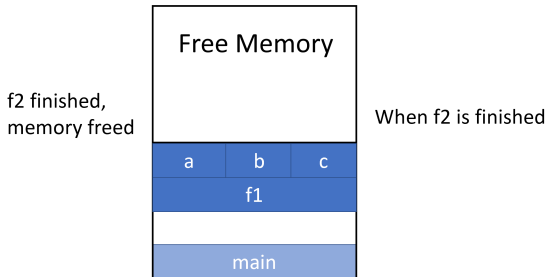
Below is the stack when f3 has finished.

f3 finished,
memory freed

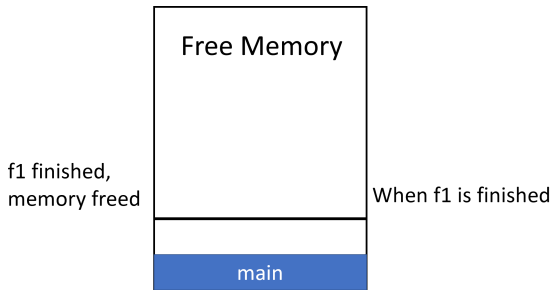


When f3 is finished

Below is the stack when f2 has finished.



Below is the stack when f1 has finished.



What if there is no base case?

```
void countdown(int n) {  
    printf("%d\n", n);  
    countdown(n - 1);  
}
```

Each call creates another unfinished call:

```
countdown(3), countdown(2), countdown(1),  
countdown(0), countdown(-1), ...
```

We need to store an infinite number of n's.

But stack memory is limited.

Uncontrolled growth can exhaust it and cause a **stack overflow**.

How does this compare with an infinite loop? E.g.:

```
int n = 3;
while(1){
    print("%d /n", n)
    n = n - 1;
}
```

Infinite loop	Recursion that does not stop
Repeats inside the current function call	Repeatedly creates new function calls
Usually keeps roughly the same call-stack depth	Usually makes the call stack deeper
Does not stop by itself	Eventually exhausts the stack

Both are control-flow errors, but they use memory differently.

A second recursive example

The recursive countdown function was void.

But recursive functions can return a value.

Example: computing the sum $S(n) = 1 + 2 + \dots + n$.

We know that

$$S(n) = 1 + 2 + \dots + n = n + (1 + 2 + \dots + (n - 1)).$$

So

$$S(n) = n + S(n - 1) \quad \text{for } n > 0,$$

where we define $S(0) = 0$.

This definition contains a **base case** and a **smaller version of the same problem**.

Translating the definition into C

```
int sum_to(int n) {  
    if (n <= 0) {  
        return 0;  
    }  
  
    return n + sum_to(n - 1);  
}
```

The current call cannot finish the addition until the smaller call returns a value.

The calls build the expression:

$$\begin{aligned} S(3) &= 3 + S(2) \\ &= 3 + 2 + S(1) \\ &= 3 + 2 + 1 + S(0) \\ &= 3 + 2 + 1 + 0. \end{aligned}$$

by growing the call stack.

The base case starts the return journey:

$$S(0) = 0, \quad S(1) = 1, \quad S(2) = 3, \quad S(3) = 6,$$

during which the call stack will unwind.

A step-by-step instrumented function

```
int sum_to(int n){  
    printf("enter sum_to(%d)\n", n);  
  
    if (n <= 0) {  
        printf("leave sum_to(0), returning 0\n");  
        return 0;  
    }  
  
    int smaller_sum = sum_to(n - 1);  
    int result = n + smaller_sum;  
  
    printf("leave sum_to(%d), returning %d\n", n, result);  
    return result;  
}
```

Let's look at recursive_sum.c.

Loop or recursion?

Loop	Recursion
Makes repetition explicit	Expresses a problem using smaller versions of itself
Usually natural for a linear sequence	Often natural for recursive or hierarchical definitions
Typically keeps one function call active	May keep many function calls active

- Countdown and `sum_to` can be written either way.
- **For large `n`, recursion is slower because it uses a lot of memory!**
- In practice one needs to consider speed, memory usage and clarity of the code.

Check your understanding

Without running the program, consider `sum_to(3)`.

1. What is the base case?
2. What makes each recursive input smaller?
3. Which frames are present when `sum_to(0)` begins?
4. Which value is returned by each call?
5. Why does the program not lose the earlier values of `n`?

Conclusion

1. Recursion means solving a problem using a smaller version of the same problem.
2. Simple recursion needs:
 - a base case;
 - a recursive case;
 - progress towards the base case.
3. Each active call has its own stack frame.
4. The call stack grows and unwinds:
 - recursive calls add more frames;
 - returns remove them in reverse order.
5. Recursion that never reaches a base case can cause stack overflow.